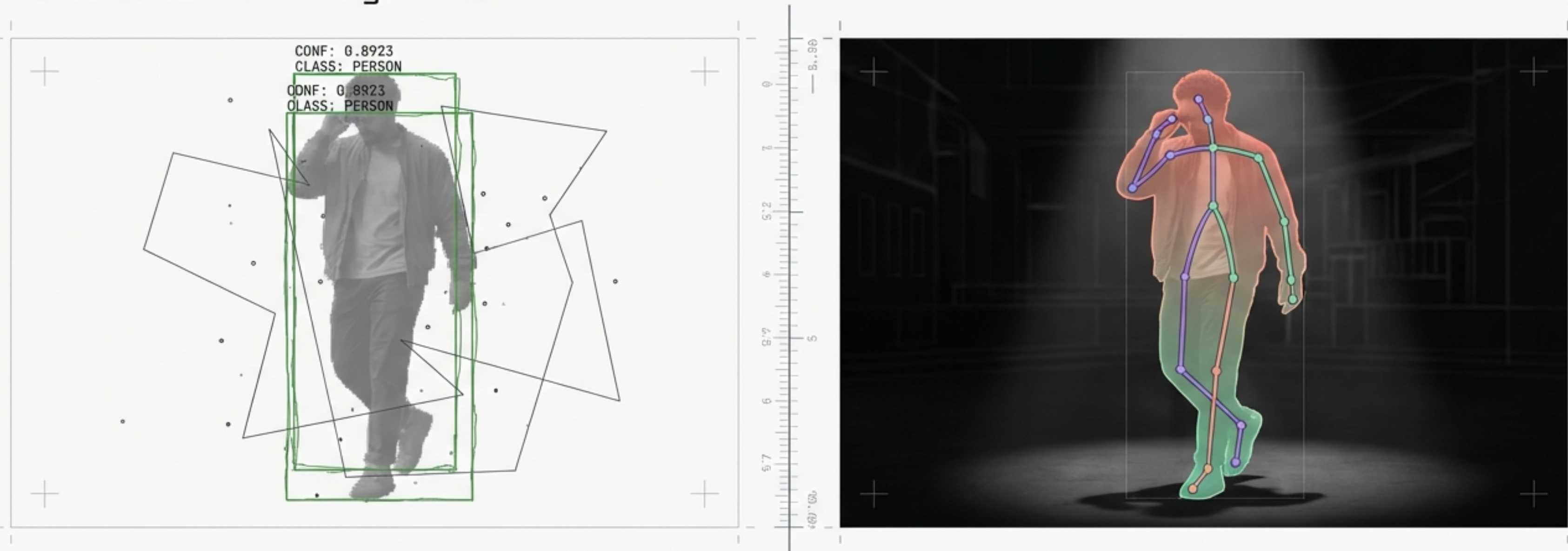


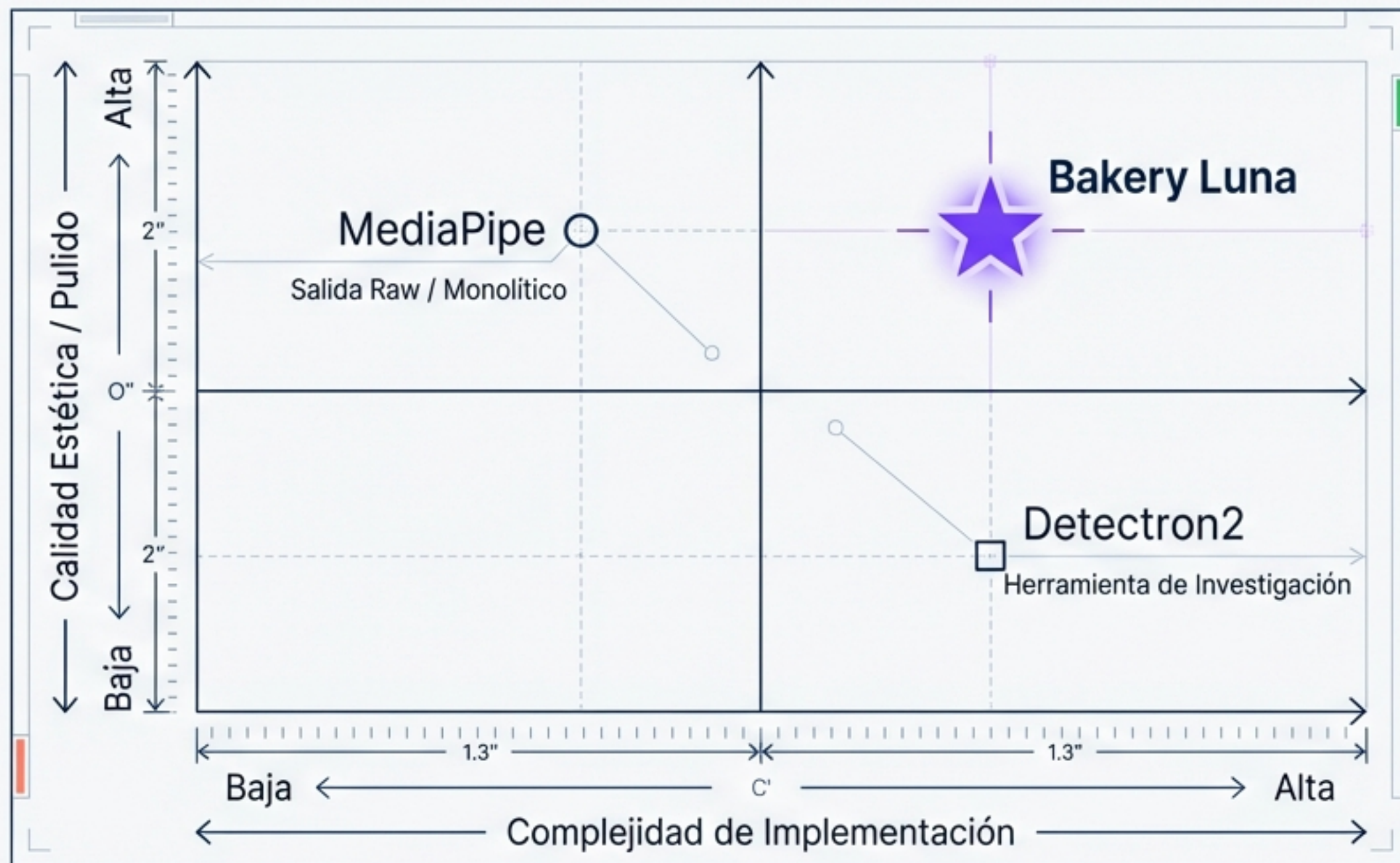
Bakery Luna: Visión Computacional Dual con Estética Cinematográfica

Un framework de inferencia modular que combina segmentación y estimación de pose con renderizado estilo “Roger Rabbit”.



El 'Sweet Spot' de la Ingeniería de Visión

Bakery Luna ocupa un nicho único entre las herramientas de investigación académica y las APIs de consumo masivo, priorizando la inferencia en producción y el acabado visual.



DIFFERENCIADORES CLAVE



Inferencia Dual:

Ejecución simultánea de YOLO-Seg y YOLO-Pose.



Hardware Consumer:

Optimización nativa para iGPU Intel (OpenVINO FP16).



Agent-Native:

Salida estructurada lista para agentes de IA.



Código Compacto: ~2000 líneas de código vs. 100k+ en frameworks de entrenamiento.

Arquitectura C4: Las 5 Capas del Framework

Core

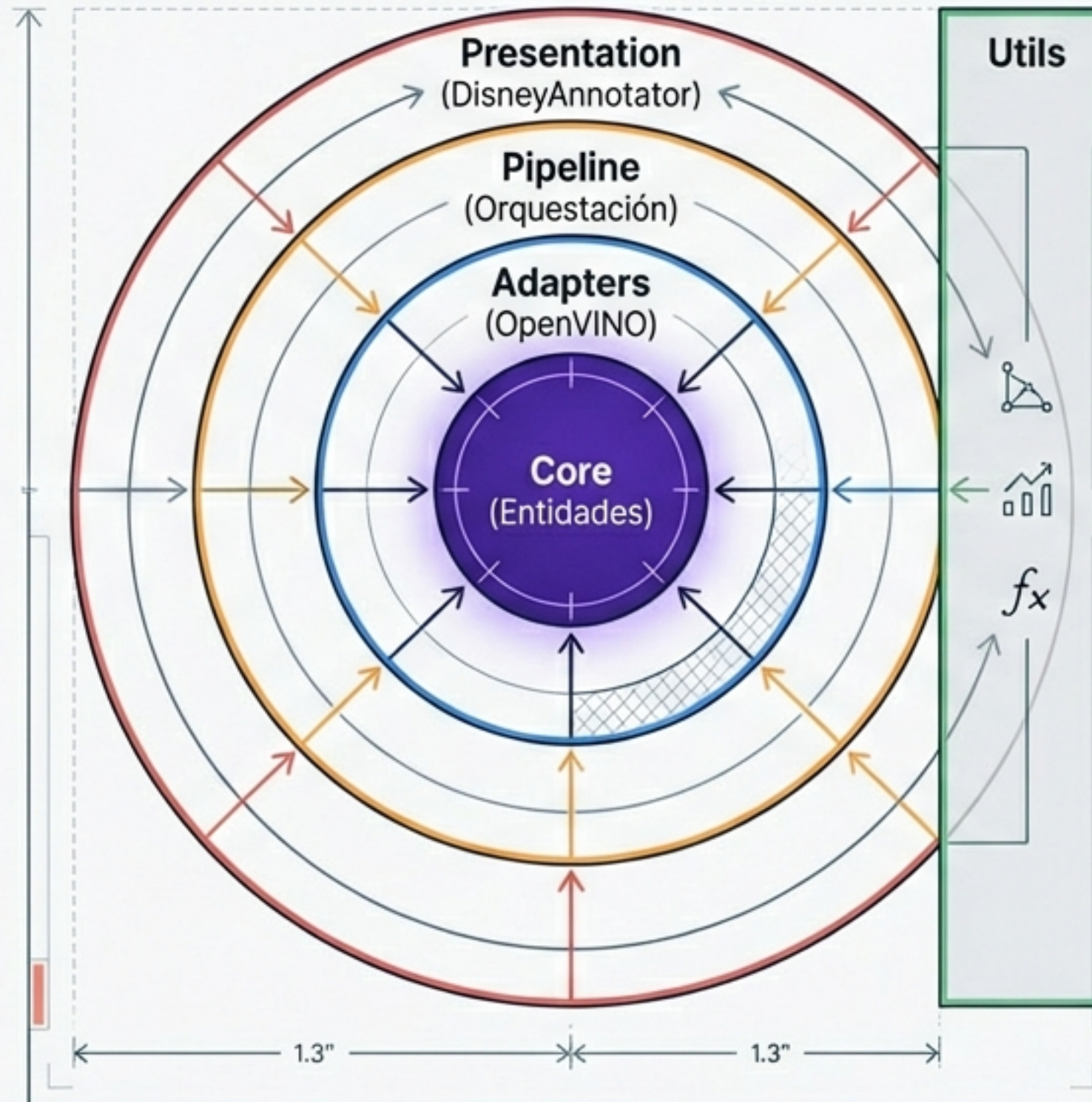
Entidades de dominio inmutables sin dependencias externas.

Adapters

Aislamiento de hardware (OpenVINO) mediante patrón Adapter.

Pipeline

Orquestación de flujos complejos (DualModelPipeline).



Presentation

Lógica de visualización separada (DisneyAnnotator).

Utils

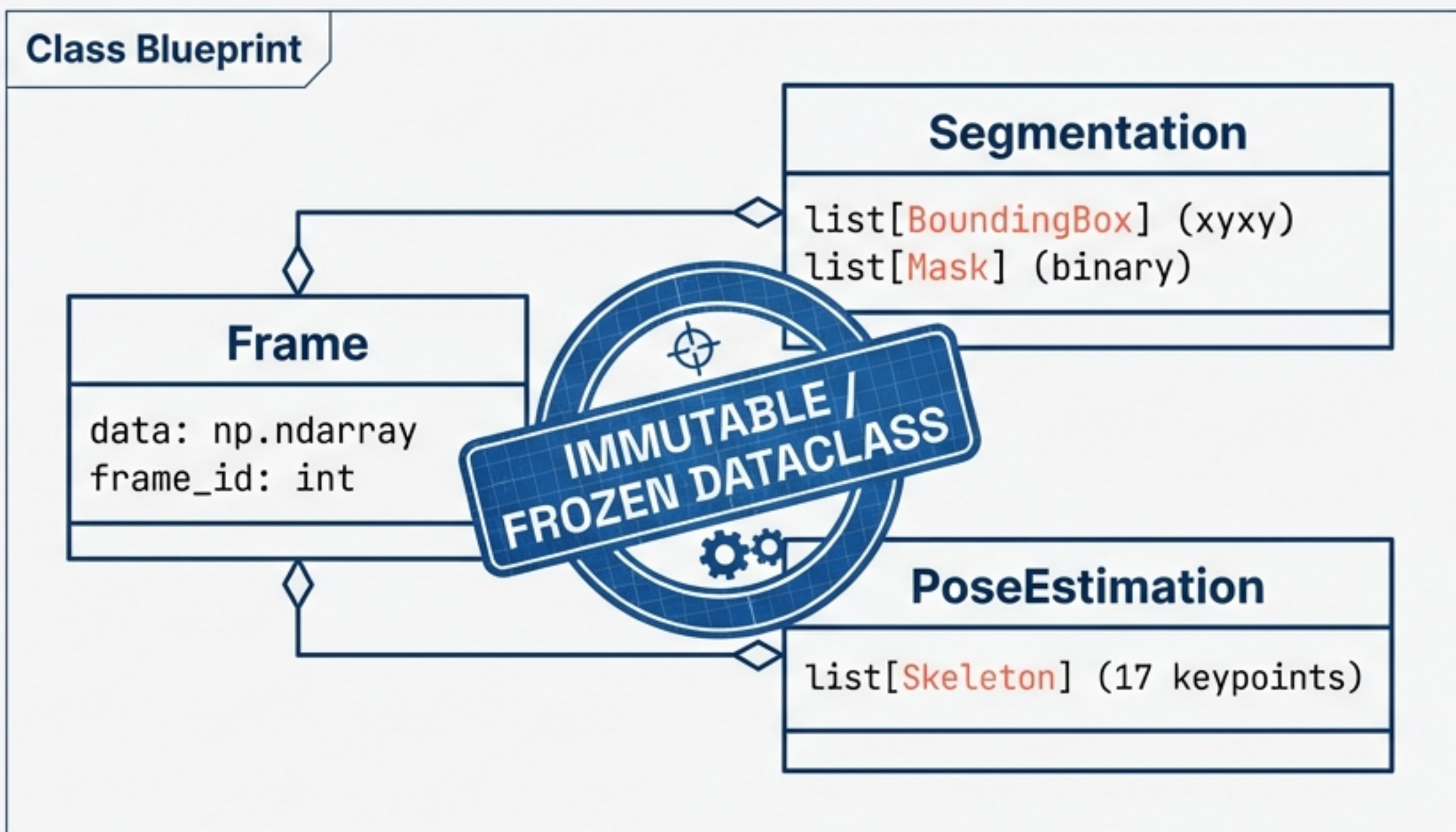
Geometría, métricas y funciones puras transversales.

****Principio de Inversión de Dependencias**:**

Los módulos de alto nivel dependen de abstracciones (Core), no de detalles de implementación.

El Núcleo: Entidades de Dominio Inmutables

Todo el sistema gira en torno a "**Value Objects**" **inmutables**. Esto garantiza que los datos no sufran mutaciones accidentales mientras fluyen por el pipeline (Thread-Safety por diseño).



✓ Key Entities List

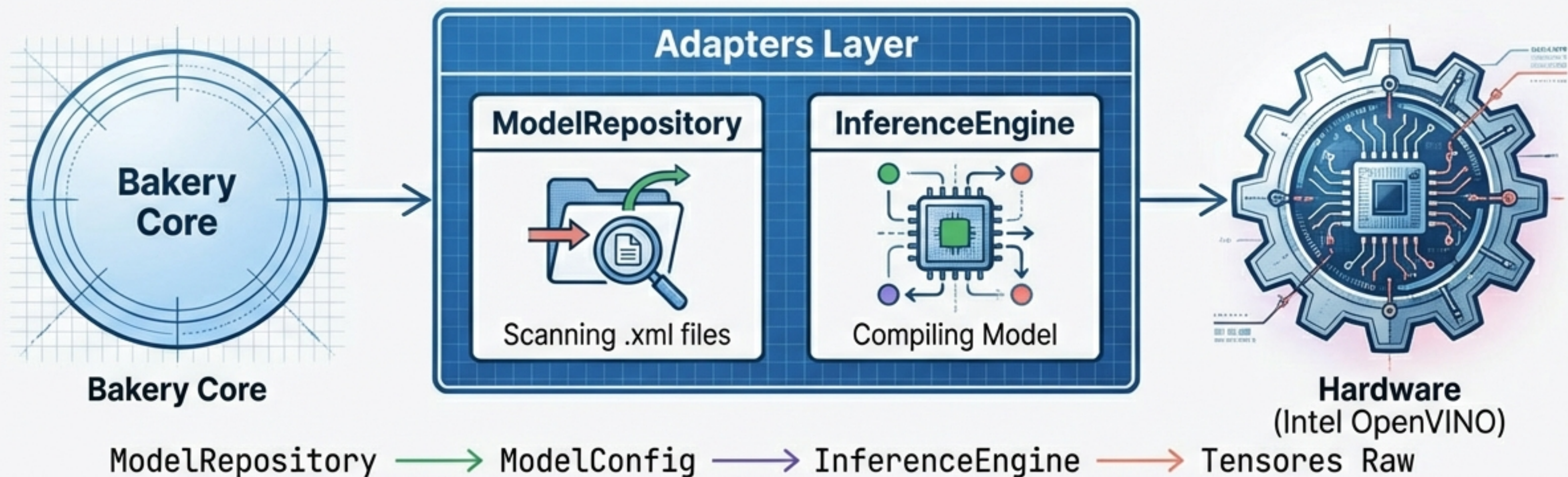
- **Frame**: El contenedor raíz (Root Aggregate).
- **Detection**: BoundingBox (coordenadas xyxy), Mask (segmentación).
- **Pose**: Skeleton (formato COCO 17 keypoints).
- **Config**: Objetos de configuración tipados (PipelineConfig).

🔴 Side Note

Las entidades se autovalidan en su construcción (`__post_init__`), fallando rápido antes de iniciar procesos costosos.

Capa de Adaptadores: Aislamiento de Hardware

El framework es **agnóstico al hardware** en sus capas superiores. La dependencia de Intel OpenVINO está encapsulada estrictamente, permitiendo cambiar el motor de inferencia sin romper la lógica de negocio.



Component Breakdown

- **ModelRepository:** Descubrimiento dinámico de archivos `.xml`. Valida formato YOLO (inputs/outputs) y extrae metadatos sin compilar.
- **InferenceEngine:** Wrapper del runtime de OpenVINO. Maneja la compilación (CPU/GPU/VPU) y la ejecución síncrona.

Sistema de Configuración y Validación

No hay 'números mágicos'. Cada aspecto del pipeline se controla mediante objetos de configuración inmutables y fuertemente tipados que establecen contratos estrictos.



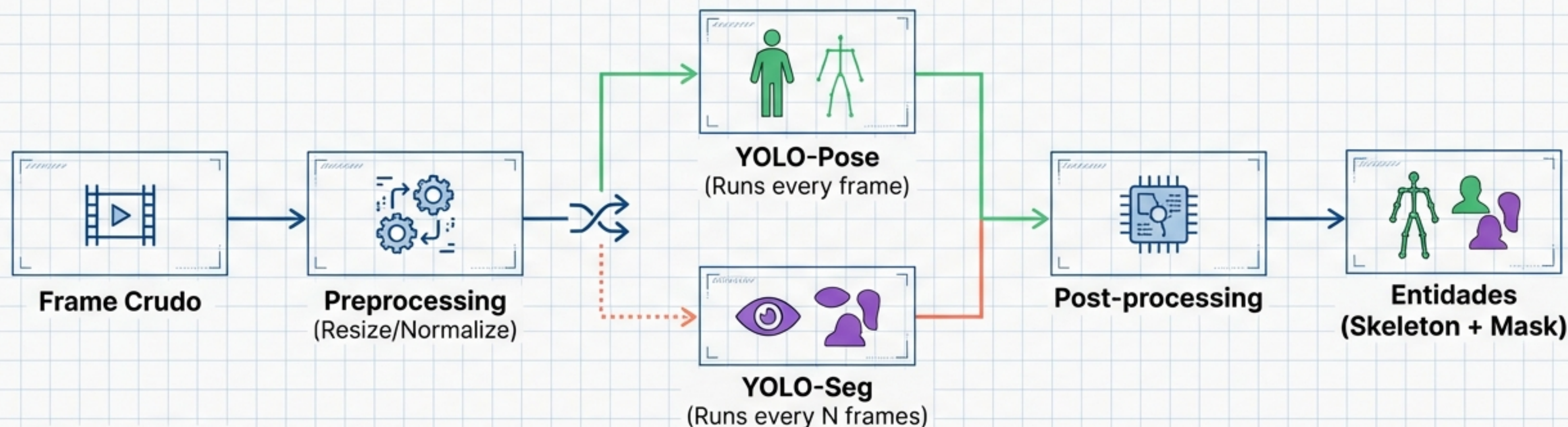
Validation Examples

- **PipelineConfig**: Controla la orquestación y filtros de clase.
- **RenderConfig**: Parametriza la estética "Disney" (niveles de oscuridad y brillo).
- **FocusLensConfig**: Define estrategias de recorte ('zoom' vs 'pad').

```
def validate_render_config(bw_darkness, spotlight):  
    if not (0.0 <= bw_darkness <= 1.0):  
        raise ValueError(f"bw_darkness must be between 0.0 and 1.0, got {bw_darkness}")  
    if spotlight < 1.0:  
        raise ValueError(f"spotlight factor must be >= 1.0, got {spotlight}")  
    # More validation logic...
```


Pipeline de Doble Modelo (`DualModelPipeline`)

El orquestador central coordina dos motores de inferencia especializados. Mientras la pose se calcula en cada frame para fluidez, la segmentación utiliza una estrategia de intervalos.

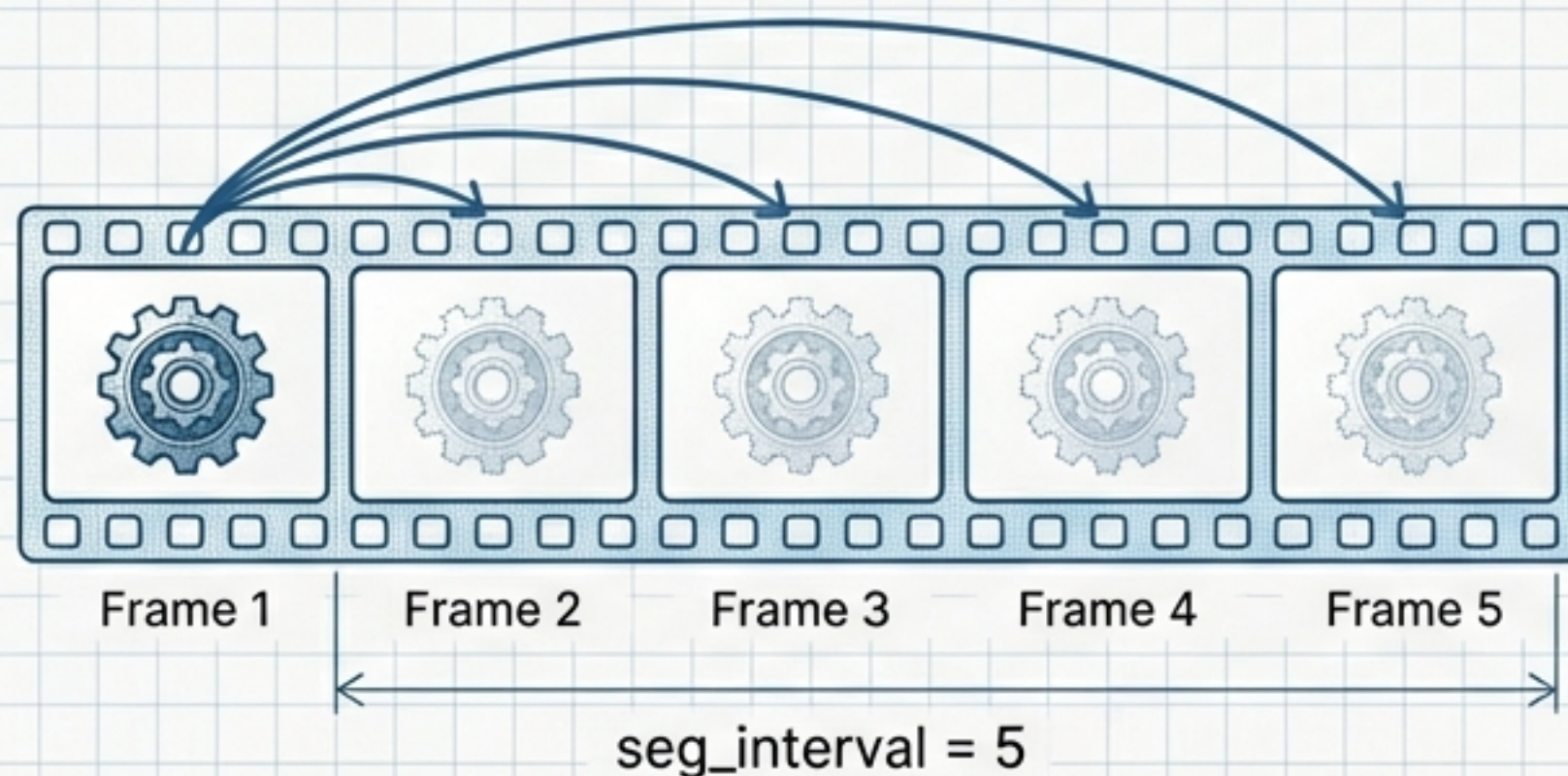


Process Flow

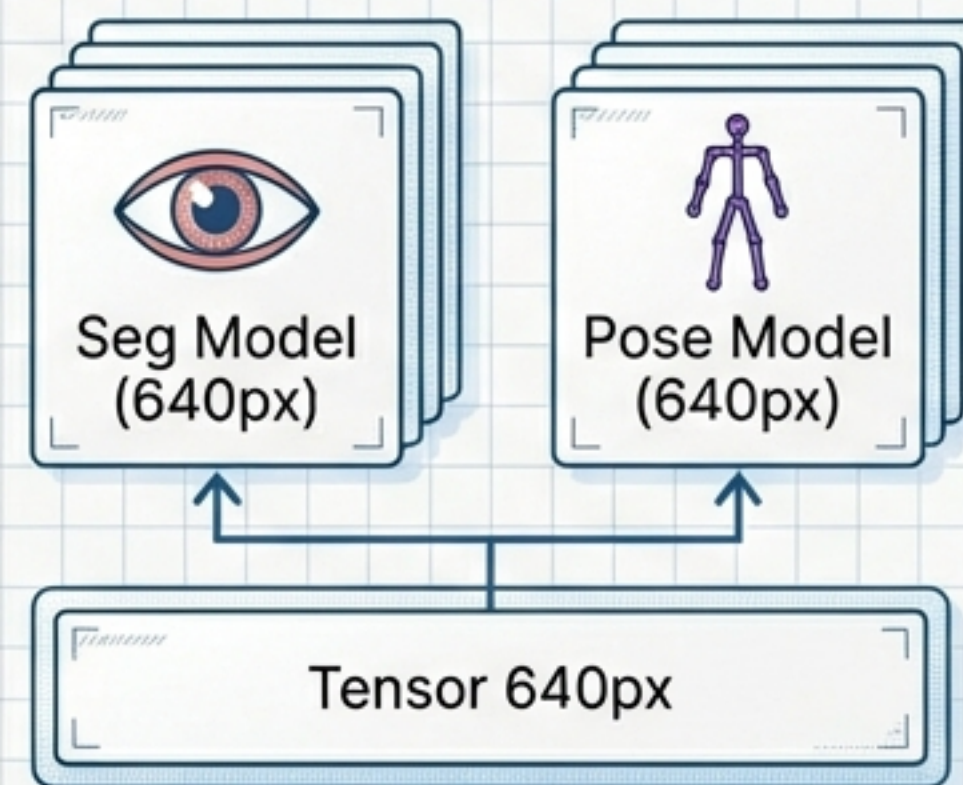
1. **Input:** Frame de video.
2. **Preprocessing:** Normalización con caché compartido.
3. **Inference A (Pose):** Ejecución continua (Alta frecuencia).
4. **Inference B (Seg):** Ejecución a intervalos (Baja frecuencia).
5. **Output:** Objetos de dominio listos para renderizar.

Optimización I: Programación Inteligente y Caché

Smart Scheduling



Preprocessing Cache



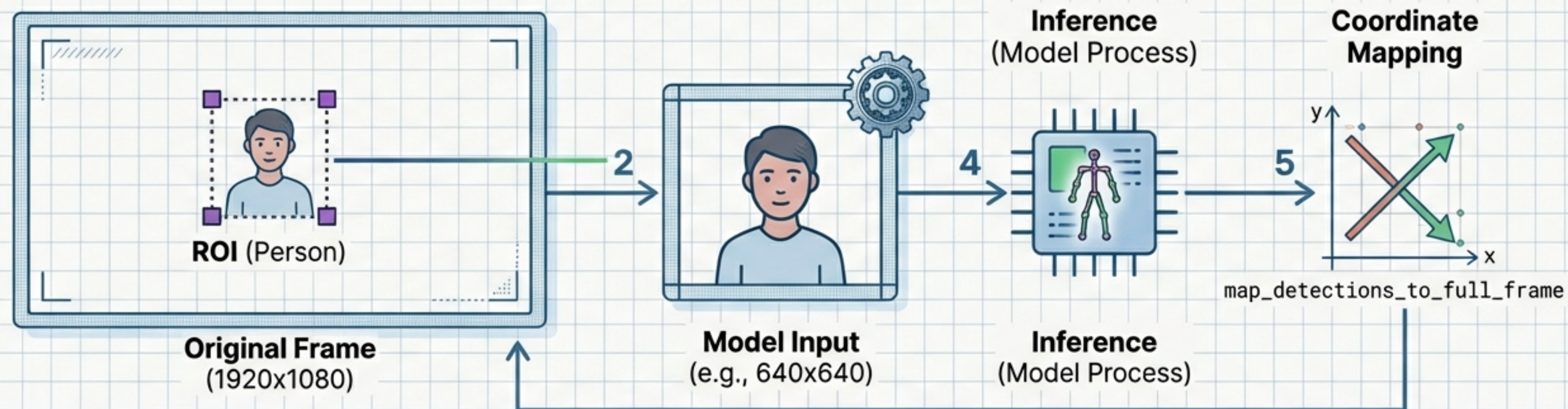
La segmentación es computacionalmente costosa.

- **Estrategia:** Ejecutar segmentación solo en frames clave (0, 5, 10...). *
- **Resultado:** Los frames intermedios reutilizan resultados previos, reduciendo la carga de cómputo en **~40%**. *

- **Lógica:** Si ambos modelos comparten resolución (ej. 640x640), el tensor preprocesado se genera una sola vez. *
- **Impacto:** Elimina operaciones redundantes de *letterbox* y normalización (**~50%** ahorro en preproceso). *

Optimización II: Sistema “Focus Lens”

Permite realizar inferencia sobre una Región de Interés (ROI) en lugar del frame completo, maximizando el detalle o reduciendo la resolución de entrada.



Strategies

- **Zoom Strategy:** Recorta la ROI y escala al tamaño del modelo. Maximiza detalle en objetos pequeños. *
- **Pad Strategy:** Agrega bordes negros si la ROI es pequeña. Evita distorsiones. *

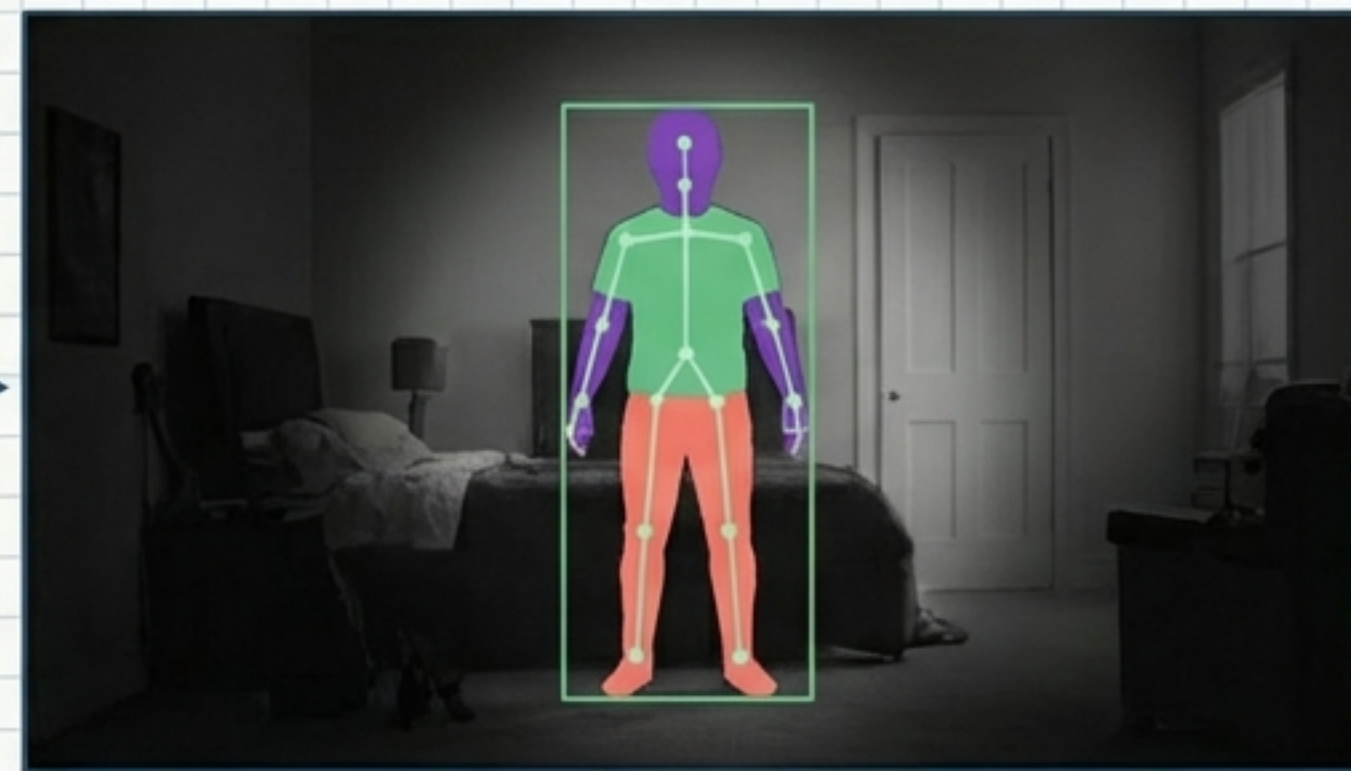
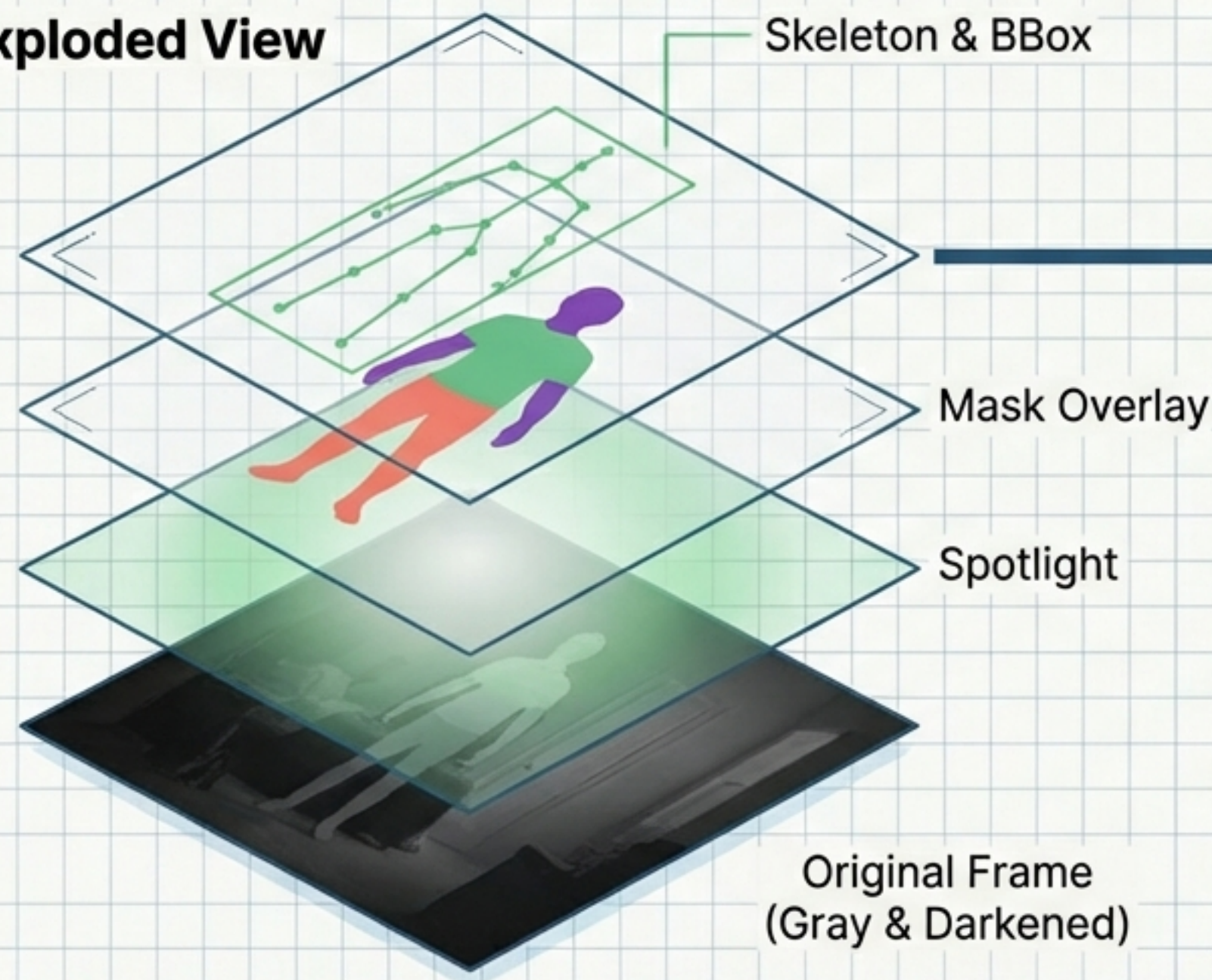
Technical Detail

Coordinate Mapping: El sistema transforma automáticamente las detecciones del espacio “recortado” al espacio original (**map_detections_to_full_frame**). *

La Capa de Presentación: `DisneyAnnotator`

Un sistema de composición de 8 capas que separa totalmente la lógica de inferencia de la visualización artística.

Exploded View



Disney Aesthetic

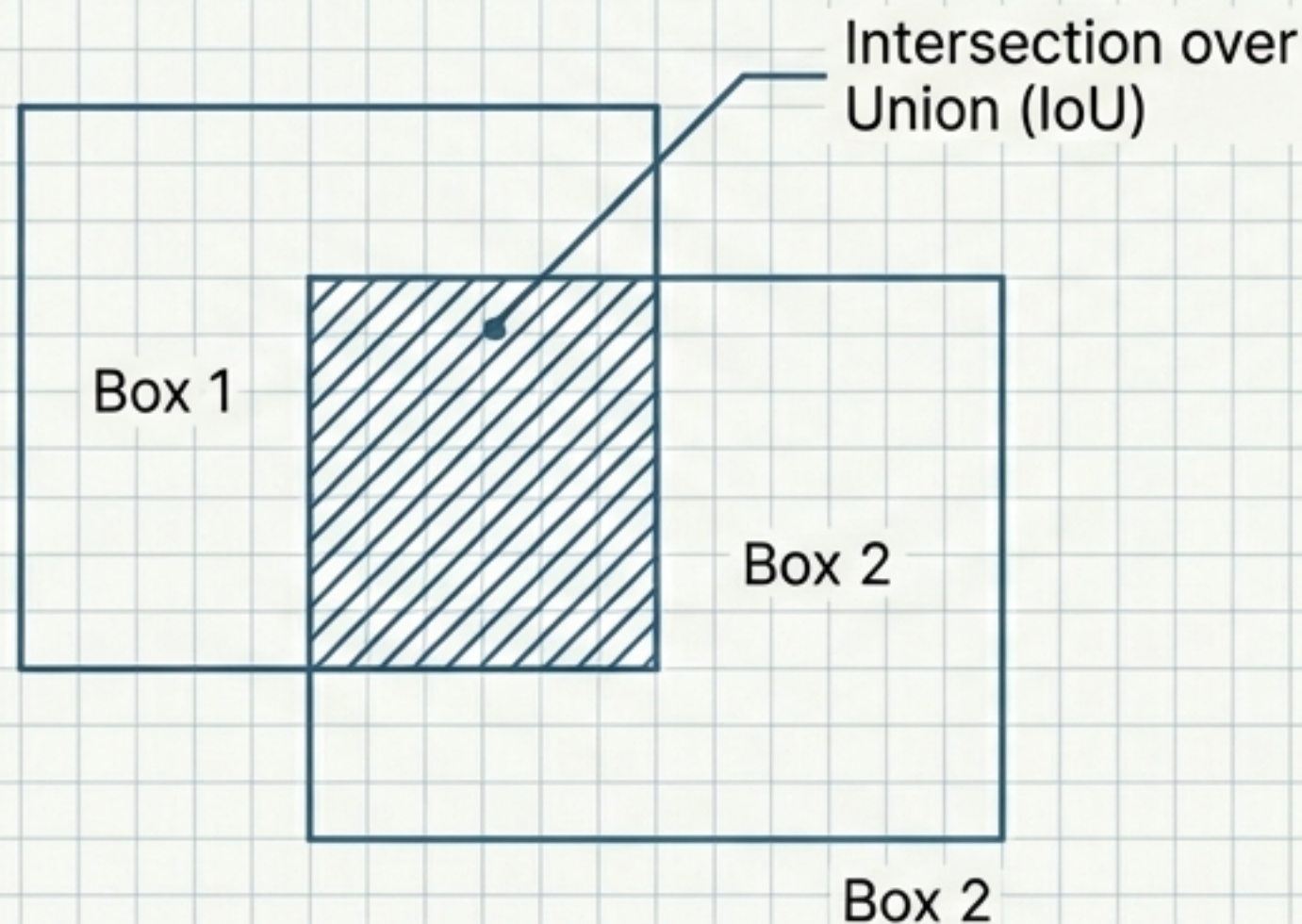
Recipe

1. **Fondo:** Desaturado y oscurecido (60% darkness) para contraste dramático.
2. **Spotlight:** Iluminación selectiva sobre detecciones.
3. **Masks:** Superposición de segmentación a todo color.
4. **Skeletons:** Estructura de pose renderizada sobre las máscaras.

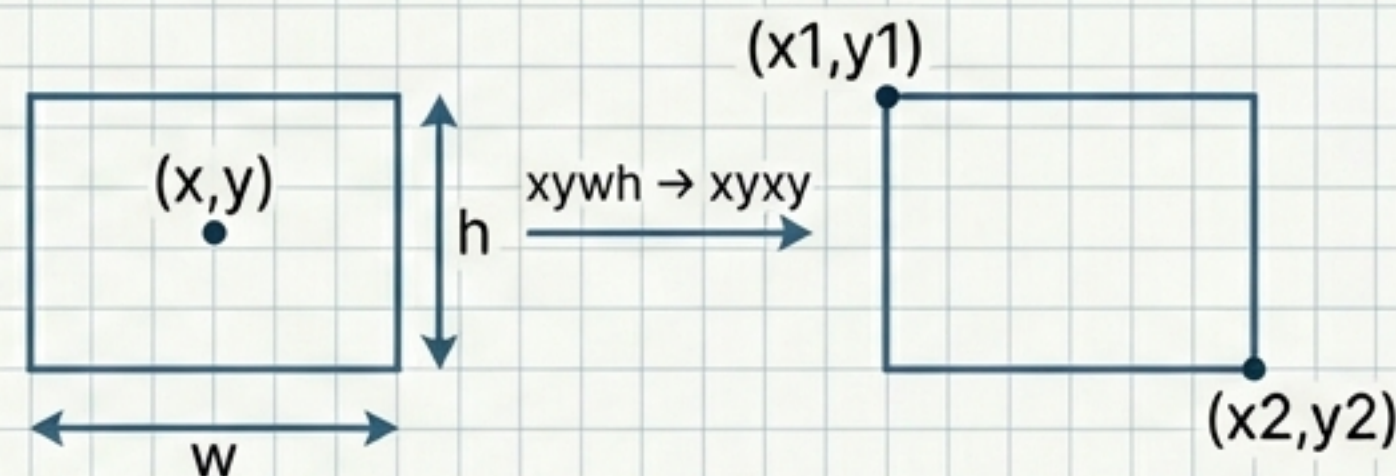
Utilidades Matemáticas y Geometría

El framework se apoya en funciones puras y sin estado para resolver problemas geométricos críticos durante el post-procesamiento.

IoU Calculation



xywh $\rightarrow$ xyxy

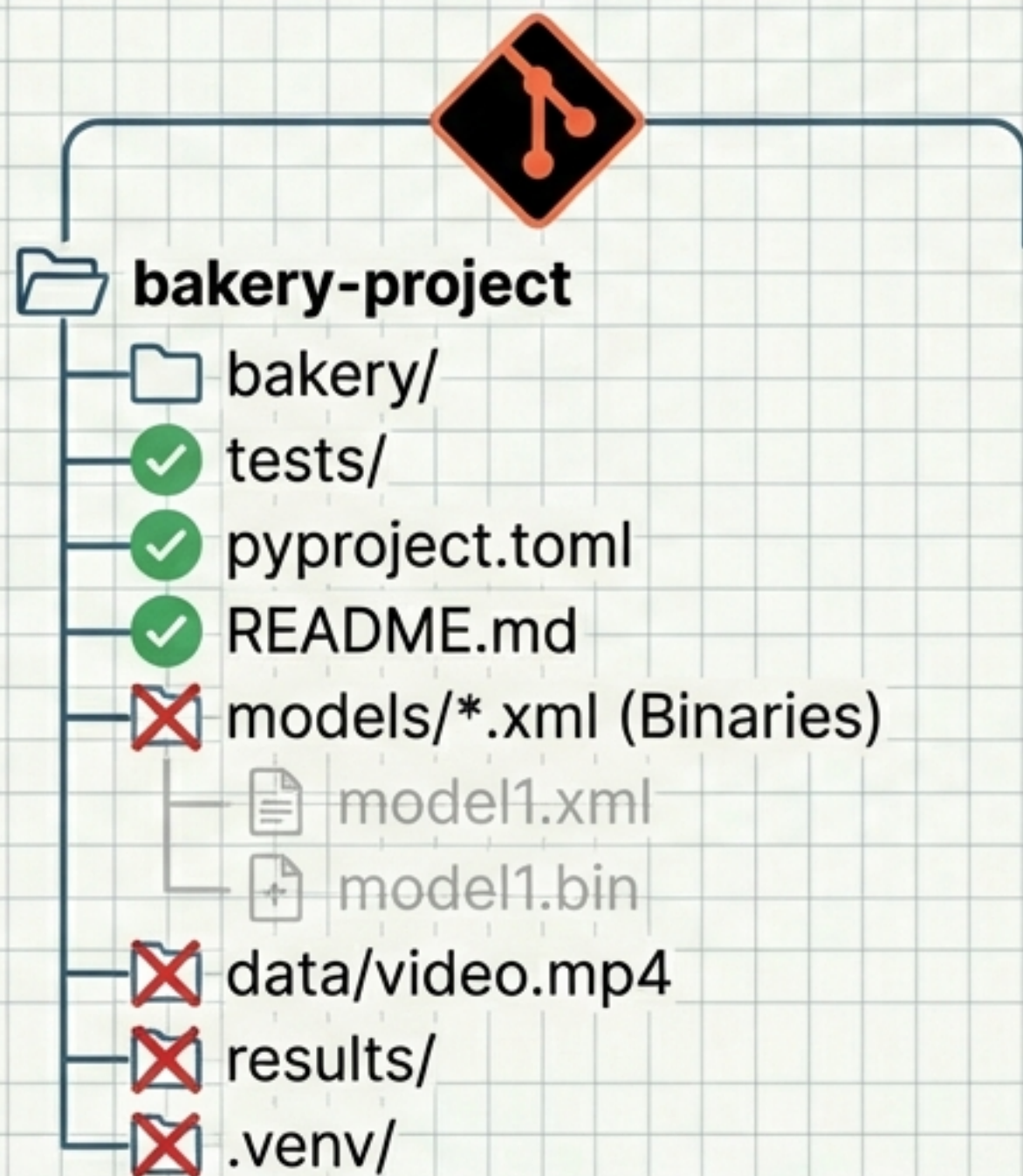


Key Functions

- **NMS** (Non-Maximum Suppression): Elimina detecciones duplicadas basándose en el solapamiento (IoU).
- **xywh2xyxy**: Conversión robusta entre formatos de coordenadas.
- **Métricas**: 'FPSCounter' y 'PerformanceMetrics' rastrean la eficiencia en tiempo real (ratio de segmentación vs frames totales).

Modelo de Desarrollo “Source-Only”

Para mantener el repositorio ligero y limpio, los artefactos binarios pesados están estrictamente excluidos del control de versiones.



Git Strategy

- **Rastreado** (Git): Código fuente, configuración, tests unitarios, documentación.
- **Local** (Artifacts): Modelos OpenVINO, videos de entrada, resultados renderizados.

Benefit

Previene el “bloat” del repositorio. Los modelos se descubren dinámicamente en tiempo de ejecución (ModelRepository).

Ejecución Rápida y CLI

El punto de entrada principal es `run\_luna.py`. Diseñado para ser intuitivo, observable y listo para automatización.

Key Commands List

- `discover\_models()`
- `create\_pipeline()`
- `process\_video()`

```
> python run_luna.py --source video.mp4 --seg-interval 5
```

```
[INFO] Models discovered: 2
```

```
[INFO] Pipeline initialized (Device: CPU)
```

```
Processing... [=====] 100%
```

```
-----  
Total frames:      1540
```

```
Segmentation runs: 308 (Efficiency: 5.0x)
```

```
Pose runs:         1540
```

```
Average FPS:      42.5  
-----
```

```
Output saved to: results/output_luna.mp4
```

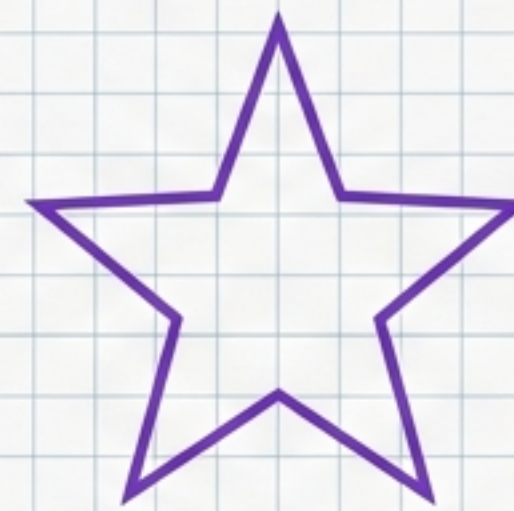

Estado Actual y Roadmap (Proyecto Juno)

Fase 1.6: Luna (Completada)



- ✓ Arquitectura Limpia implementada.
- ✓ 126 tests unitarios pasando (100% cobertura en Core).
- ✓ Listo para Producción.

Fase 2: Juno (Sigüientes Pasos)



- Soporte para múltiples cámaras simultáneas.
- Inferencia Asíncrona (Paralelización real).
- Integración de Tracking para coherencia temporal.



Conclusión: El Arte de la Ingeniería

"Bakery Luna demuestra que el código de visión computacional puede ser tan elegante como los resultados que produce."

✓ Simplicidad (~2000 LoC) ✓ Rendimiento (iGPU Optimized) Estética (Cinematográfica)